

Homework 5

GWU CSCI 1112 - Spring 2026

March 26, 2026

1 Introduction

This homework is intended to reinforce your understanding of working with Stacks, Queues, and overall application and algorithm development. Your task is to develop an implementation that solves the problem described using the provided framework.

1.1 Evaluation Date

April 3, 2026 at the beginning of your scheduled lab

1.2 Evaluation Method

Handwritten programming quiz where you will need to recount in pseudo-code aspects of this programming homework. The manual programming tasks will be relatively simple, but they will require a student to have practiced the homework to immediately understand and answer the question. Students who have not practiced or at least studied this homework, will generally be guessing during the quiz and they will typically require a long time to complete the quiz. Quizzes are timed and that time is capped at 30 minutes for all students including DSS students as this quiz involves recitation of already learned and practiced concepts and is not asking for full creative thinking. *Someone who has done this work should be able to finish the associated programming quiz in less than 10 minutes. If you have a handwriting disability, please talk to the professor before the evaluation date.*

1.3 Grading Rubric

I leave this here as an artifact for your reference. This is how this homework was graded when we evaluated student implementations. A rubric was provided with each homework and as the homeworks progressed, the percentages would encompass more critical metrics.

`UnitTests.java` AND `UseCases.java` must both compile otherwise the assignment will receive a zero grade.

- 30 % Implementation
- 25 % Passing all unit tests (5 % each block)
- 30 % For sufficiency of comments (10% for File Headers, 10% for Function Headers, 10% for Inline)
- 15 % Consistency of coding style

1.4 Comments

This section applies to graded student implementations. It is an artifact left for reference so you can understand expectations in the framework.

In this and future homeworks, you will have increased responsibility with commenting of code. Documentation is as critical a part of a programmer's job as programming code. In the previous frameworks, we have provided ample

documentation for every file as a model for proper documentation procedures. It is your now responsibility to properly document your implementations in their entirety. This amount and type of documentation is an industry standard and we have provided sufficient examples in all of the assignments so far for you to base your work on.

In this and any future assignments, you will be expected to include the same types and detail for your documentation. There are three classes of comments:

- File header comments
- Function header comments
- Line comments

File header comments summarize the content of the file so that a programmer can quickly understand what is in the file without scanning through the file itself, provide clear natural language descriptions that can be matched by text based file search utilities, and associate authors, organizations, and copyrights with the code.

Function header comments document how to use functions in detail. The style used in this class is defined by the [Doxygen Standard](#) which facilitates the automatic generation of html web pages from code. All functions must have a comment that precedes the function declaration, *i.e.* must be before the function so that it clearly stands out, and this comment must describe 3 things when applicable: what the function does, every input into the function, and what the function returns if anything.

Line comments have a variety of roles. Typically they are used explain the internal logic within a class and often are associated with descriptions of the flow of execution inside a function. However, they are also associated with instance variables and explaining their role in the class, so they are not exclusively associated with functions. Line comments should be written in plain English and should add information that is not obvious from the code itself. Good line comments should help a reader understand the program even if that person does not understand the language or programming in general. Line comments are not necessary for every line of code, and can be used in groups to explain several steps grouped together. In general, line comments should be left justified to the same indentation as the current coding block and they should precede the upcoming code to prepare the reader for the code they are about to read.

1.5 Use of Imported Java Types

A goal of this class is to develop an understanding of how to build data structures. We will be developing our own classes for ArrayLists, LinkedLists, Stacks, Queues, BinaryTrees, etc. For this reason, you are not permitted to import any of the existing classes from any library unless explicitly stated.

2 Banking & Investments

Banks remain one of the largest and longstanding collection of organizations in the United States. As of 2021, there are 4,236 commercial banks with 72,166 branches across the 50 states ([source](#)). The banking industry as a whole dates all the way back to 1791; only 15 years after the Declaration of Independence was signed!

Each bank in the United States serves customers and consumers independently and normally provides a rather unique perspective and background to assist customers. As you can imagine, 72,000 locations each performing tens to millions of unique transactions, filings, and paperwork each day, would be a lot to manage and continue developing and maintaining for over 200 years. Obviously, software has been developed to assist in this management. However, more than 4,000 pieces of software is a lot to build, manage, and maintain.

This homework will have you build a fundamental banking system. Your banking system is very similar to the systems that are actually used today across the country. This system can scale to any number of banks across the country, so think wisely when making your design decisions. In practice, this system can support a number of transaction types, numerous users of the bank, as well as the systems to support a backlog of responsibilities for daily operation.

2.1 Design

Fundamentally, every bank consists of two elements: Bank Accounts, and Transactions that use those accounts.

Each account has 4 operations that can happen:

- 1) Open the account
- 2) Close the account
- 3) Deposit money into the account
- 4) Withdraw money from the account

Every interaction with an account in any of these 4 ways is referred to as a transaction. Each transaction has 2 main parts: 1) the interactions between teller/banker and customer indicating the action and amount, and 2) the modification of a bank account on behalf of the customer's request. Consider the creation of a new transaction as part 1, and processing the transaction itself to be part 2.

Over the course of a day, these transactions are on a "first come, first serve" basis. As expected, if we show up to the bank at the same time, whomever enters the line for a teller first is tended to first and their transactions are initialized before the other, so order is critically important. From the bank's perspective, these transactions might take time to process, but each transaction must be fulfilled in the same order that it came into the bank to maintain the integrity of the order of transactions. At the end of the day, if there is a backlog of modern transactions to be processed, the same "first come, first serve" ordering must be maintained.

Since banks have a long history, there are often times situations in which tickets from previous days/weeks/months/years' bank transactions must be logged into the system and the balances of said accounts updated to reflect the transactions that were previously thought to be lost. Banks will downsize, branches of the same bank will close, liquidations happen, and banks will buy other banks completely and have to make all of the services uniform. For example, when a computer system goes down, transactions may still occur without perfect computer based authentication, and thus must be verified and authenticated after the fact to moderate the correct balances and transactions from the day.

Each transaction will log one of the earlier mentioned operations: opening an account, a withdrawal, a deposit, or closing an account. You will see in `Transactions.java` an instance variable `operation` which will indicate the type of transaction.

2.1.1 Bank Accounts

Each time a new account is created, the user must provide their name, and in doing so, their balance will be zero, and their account number is created. For the sake of simplicity, the account number details are specified in the framework section.

The bank must keep track of all accounts that are created, so assuming a fixed number of accounts for the bank would create many issues. Even in the modern era, banks can support a dynamic number of accounts at one time.

2.1.2 Transactions

As mentioned earlier, transactions can log any of the 4 operations: opening an account, closing an account, a withdrawal, or a deposit. For a real bank, these can include cash withdrawals or deposits, checks, online payments, debit card charges, wire transfers and loan payments. For this exercise, we will be focusing on the withdrawal and depositing actions for a bank account, ignoring the source of the money.

Here is an example of what some transactions could include. This is not what will print out for your homework, rather it shows the logic of bank transactions:

```
// Sample Transactions
Create Account - Name: 'A'
(Balance : 0 - Account ID : 65)

Check Balance - Account ID: 65
:: Balance: 0

Deposit Money - Account ID: 65 - Amount: 1042

Withdraw Money - Account ID: 65 - Amount: 101

Check Balance - Account ID: 65
:: Balance: 941

Close Account - Account ID: 65
```

2.1.3 Creating a Bank Account

Your bank will manage creating new bank accounts, which only requires the name of the customer who would like to open the account. Since you do not know how many customers will be using your bank, it is recommended that you use a data structure that can dynamically size based off of the demand of the customer base. The more users who use your bank, the more likely it is that you underestimate the volume of bank accounts.

2.1.4 Deposits

Once a customer has an account, then they will be allowed to deposit money into the account. When you provide account number, provide the money to deposit, then the bank will increase the balance of the account to reflect the money you just deposited into your account.

If the account number provided is invalid, then no money can be deposited to begin with, so the transaction is processed, but no money is actually deposited.

2.1.5 Withdrawals

In order to withdraw from an account, a customer must go to the bank, request to withdraw money from their account, and provide the amount to withdraw. Some banks will allow you to withdraw all of your money, and most banks support over drafting from your account, which actually allows your account balance to fall to a negative amount. You must withdraw the money for the customer even if they do not have the account balance, yes you will support negative balances.

2.1.6 Checking Balance

When you go to the bank, you are allowed to query the balance of an account that you own. In this case, since this is just querying information without modification, whenever you provide a valid account ID, you can check the balance of an account. When you provide the account ID, the bank will give you the value of the current balance of an account without modifying the balance of the account itself. If your account is not found, then there will be a negative value returned to your customer to indicate that the account number was not found in your system.

2.1.7 Other Transaction Type

Although you are designing a modern banking system, you must be able to support transactions that may have different formatting due to the last system, the age, or just errors in general.

If the operation of the transaction is not one of the above transactions, you must assume that the transaction is old and not in the same format as modern systems. When the transaction is created in your new system, you may have to call a helping function to translate from the old transaction type into one of the 4 above transactions in order to process the information successfully.

We typically use “infix notation” to represent mathematical equations meaning a binary operator, *i.e.* an operator with a “left” and “right” operand, appears in between the two operands. For example, we read $100 + 5$ similarly to how we think about the math. “100 plus 5 equals 105”. However, this is not the only notational way to represent a mathematical formula.

Commonly referred to as “postfix notation”, banks wrote both operands first before a binary operator. The infix equation $a + b$ would therefore be $ab+$ in postfix notation.

This seems like a novelty, but postfix computing was extremely widespread up until the modern general computer replaced many specialized computers around the turn of the century. You would find this type of math used in cash registers and calculators because postfix calculation can be done entirely on a simple stack while infix calculation requires more computing power and memory to parse the correct order of operations.

Post fix notation changes the order, but not the operations. For example, $100\ 5\ +$ is read in as “100, 5, add the operands together” which still equals 105.

We can extend this to more complex math expressions for banks when processing multiple withdrawals and deposits that will be logged on one ticket. Open an account, deposit \$100, withdraw \$10, withdraw \$75, withdraw \$2, deposit \$12 resulting in the calculation $0 + 100 - 10 - 75 - 2 + 12$. We can see that this evaluates to \$25 left in the account. Likewise, the transaction ticket from before computers reads as $0\ 100 + 10 - 75 - 2 - 12 +$. The account balance, the money requested, deposit/withdraw the money.

In a case of PEMDAS, and how to support multiple transactions on one log, you can read more about [postfix notation here](#).

These are just common examples. Banks have logs of employees recording all operations of addition, subtractions, multiplication, and division. You must be able to support all operations in case you find old slips that have written this out.

2.2 Framework

There are a number of classes, assets, and methods provided to support this program and you will only need to write three specific methods for your solution. This section will detail all of the existing elements of the framework that are there to support your program and their roles as well as the specific methods that you are to implement. A detailed explanation of the methods you need to implement will be provided in the next section of this document.

The files distributed in this framework are `Account.java`, `Stack.java`, `Transaction.java`, `Queue.java`, `Bank.java`, and `UnitTests.java`. Additionally, `Account_Solution.java` and `Account_Framework.java`, `Queue_Solution.java` and `Queue_Framework.java`, and `Stack_Solution.java` and `Stack_Framework.java` files have been provided where the files with filenames ending with `_Solution.java` contain examples of our solutions and the files with filenames ending with `_Framework.java` are copies of the original files in case you need to refer back to the initial codebase.

Numbers stored as strings cannot be used directly in a mathematical process and instead need to be explicitly converted into numerical values to be used in mathematical contexts in Java. To convert a string `str` containing a decimal value into a compatible double value for use in a math expression, you may use the built in Java operation `Double.parseDouble(str)` which parses a numeric string into a double primitive value.

Numeric double representations can be converted to strings using the built in Java operation `Double.toString(value)`. This operation takes a primitive double as the input `value` and returns a `String` object containing the converted value.

2.3 The Account class

This class serves as the most fundamental class of the banking system. Since transactions rely on an a single account, and banks rely on multiple accounts, there is no way to interact with customers without successfully creating accounts that the banks can use.

Given the name of your customer, you must implement the *parameterized constructor* that assigns values to the 3 instance variables of the class. The `accountName` should be the value assigned to the constructor's `name` parameter, the balance is set to 0 initially, and the account number is the integer sum of the ascii values in the account name.

2.4 The Stack class

This is created for you to implement and use a stack for your bank transactions. Specifically, this is here to reinforce the clearly stated policy that you will not be using an external java library to accomplish your homework.

As expected, this class supports the expected operations of the stack ADT: `push`, `pop`, `isEmpty`, and `size`.

Notice: This framework assumes use of the `StackElement.java` class. Think about the implementation concepts covered in the previous homework for lists. Since we interact with stacks at one end of a list, it should be obvious what type of structure we should use that support dynamic sizing, and consistent $O(1)$ insert/remove functions regardless of stack state.

Hint: Postfix notation is very easy to process when a stack is leveraged.

2.5 The Queue class

This is created for you to implement and use a queue for your bank. Specifically, this is here to reinforce the clearly stated policy that you will not be using an external java library to accomplish your homework.

As expected, this class supports the expected operations of the queue ADT: `enqueue`, `dequeue`, `isEmpty`, and `size`.

Notice: This framework assumes use of the `QueueElement.java` class. Think about the implementation concepts covered in the previous homework for lists. Since we interact with queues at 2 data points, we have the option to choose the implementation that allows fast insert and remove functions at 2 locations.

Hint: This will help manage the order of transactions.

2.6 The Transaction class

This class serves to manage an individual transaction. Given an account number for the transaction, the operation of the transaction and the content for the transaction, the purpose of this class is to establish a uniform system for transactions at your bank.

```
public Transaction(int acctNumber, int operation, String content)
```

Each transaction has 4 possible operations:

- 0 - Open / Close Account
- 1 - Deposit

- 2 - Withdraw
- Other - Old Transaction

In the case of Operation 0, 1 or 2, the operation is logged as is, and the content is assumed to be the value to withdraw/deposit, or 0 for opening/closing account and should be saved as a double.

In the case of any operation outside of $[0, 1, 2]$, this indicates that the transaction most likely comes from the old system, and required a different format. You must translate this content into the model of today's standard.

You are provided the framework for `translateOldOperation(String content)` which should be used to translate from the old post-fix notation into one of the above operations. The function will parse the content of the logged transaction, calculate the amount, and store into the transaction's `amount` instance variable. It will then determine whether the transaction should be a deposit or withdrawal reflecting a positive or negative amount.

To parse a `String content` separated by spaces, *i.e. space delimited*, you may use the `String` class built in `split` method. For example, given the following:

```
String content = "Hello World"
String[] contentArray = content.split(" ");
```

`contentArray` will contain `["Hello", "World"]` after `content` is split on the space delimiter.

Remember that `translateOldOperation` is converting old transactions expressed in postfix notation into a single total transaction value that can then be applied to an account, so its goal is to compute a total value from a complex postfix expression. If this hint does not suggest to you how to support this process, then re-read any sections discussing postfix notation above as the implementation is openly stated in those discussions.

2.7 The Bank class

This class serves as the overall bank for this model. Your bank will contain multiple `Accounts`, and will serve to create, close, and interact with specific accounts. If an `Account` doesn't exist, then this class serves to protect your customers from acting maliciously and withdrawing from accounts that are not actually created in your system. This class will also serve to manage transactions from the current and the old banking systems in one format.

`int createAccount(String name)`

Given the name of a customer of your bank, you must create an account for them. Since your bank must allow for an indeterminate number of accounts, you must support a dynamic size of accounts stored in the bank. At the end of this function, you must return the account number for the customer so that they may use it for all other operations at your bank.

`double closeAccount(String name)`

Given the account number, you must close the account in the bank. This will require searching for the account number in the total list of accounts at the bank, and if found, remove the account, and return the remaining balance to the customer. If the account was not found, you must return a -1 to indicate that the balance did not exist. Even if the customer provided the wrong account number by accident, you don't want to provide them with a remaining balance that shouldn't be theirs.

```
double getBalance(int accountNumber)
void deposit(int accountNumber, double amount)
void withdraw(int accountNumber, double amount)
```

These three functions serve similar purposes. You must find the account, and either query the balance, deposit the amount into the account, or withdraw from the account.

```
public void addTransaction(Transaction t)
```

This method serves to add a Transaction to the bank's list of total transactions. Calling this function will not require you to act upon the transaction, simply adding the transaction to the list of transactions yet to be processed.

```
void processTransaction()
```

This method will remove the next transaction from the list of transactions and will subsequently act upon the operation of the transaction. In the case of deposit or withdraw, this method should be logically simple.

In the case of operation 0, opening or closing an account, you cannot open an account that already exists, and must only close accounts that are currently in the system. So, this will require you to search your accounts and compare the accounts in the bank to the account number listed on the transaction. If the account does not exist, you must open the account, and if the account does exist, you must close that account.

If you must create an account, make the account name reflective of the ascii character of the account number listed in the transaction.

2.8 The UnitTests class

This class, as all other homework assignments, serves as a supporting file to help your development. This class may not be complete, and thus requires you to implement the functions as specified in this document as well as running the use cases file to ensure that your bank is working correctly.

This class has 5 blocks to represent the 5 classes you must implement, and in a similar order to how they are listed here.